

Project Description

The Point System Database is a robust solution designed to efficiently manage the transportation services of an organization, such as a school. This system offers a centralized platform to manage users (students, drivers, staff, and admins), bus information, route management, student registrations, attendance tracking, feedback collection, maintenance management, incident reporting, notifications, payments, and salary management. By leveraging a relational database (MySQL), the system ensures data integrity, efficient querying, and secure data storage.

Abstract

This project presents a comprehensive Point System Database designed to streamline transportation management for an organization. The database is developed using MySQL, ensuring efficient data management, scalability, and security. The database covers user registration, bus management, route definition, student registration, attendance tracking, feedback collection, maintenance logs, incident management, notifications, payment tracking, and salary management. The solution enhances operational efficiency, user satisfaction, and data integrity.

Objective

The objective of this project is to design a scalable, secure, and efficient Point System database that can manage all aspects of an organization's transportation services, including:

- * User management (students, drivers, staff, admins).
- * Bus management (details, drivers, capacity).
- * Route management (routes, stops, sequences).
- * Student registration for bus services.
- * Attendance tracking for students and drivers.
- * Feedback collection and management.
- * Maintenance management for buses.
- * Incident reporting and management.
- * Notifications for users.
- * Payment tracking for students.
- * Salary management for staff.

Dataset Overview (Detailed)

The dataset for this project comprises several interconnected tables that efficiently store and manage data.

Stops: Contains all available bus stops, each with a unique ID and name.

stop_id	stop_name
1	Stop A
2	Stop B

Users: Manages user details, including students, drivers, transport staff, and admins.

user_id	name	email	role	phone
1	John	john@example.com	Student	1234567890

Buses: Stores information about each bus, including capacity and driver assignments.

bus_id	bus_name	location	max_capacity	driver_id
1	Bus Alpha	Depot A	50	3

Routes: Defines the available routes, including start and end stops.

route_id	route_name	start_stop	end_stop
1	Route 1	1	3

Student Bus Registrations: Records students registered for specific buses.

registration_id	student_id	bus_id
1	4	1

Attendance: Tracks student and driver attendance.

record_id	user_id	attendance_date	status
1	4	2025-05-08	Present

Feedback: Manages student feedback for bus services and drivers.

feedback_id	student_id	driver_id	bus_id	comments	rating
1	4	3	1	Very good ride	5

Maintenance Logs: Records maintenance activities for buses.

log_id	bus_id	description	maintenance_date	cost
1	1	Oil change and brake check	2025-05-07	150

Incident Reports: Manages reports of incidents occurring on buses.

report_id	reported_by	bus_id	description	status
1	4	1	Driver was	Open

			late.	
--	--	--	-------	--

Notifications: Manages alerts and messages sent to users.

notification_id	user_id	title	message
1	4	Bus Delay	Your bus will arrive 15 min late

Payments: Records payments made by students.

payment_id	student_id	amount	payment_date	status
1	4	200.00	2025-05-08	completed

Salaries: Manages salary payments for staff.

salary_id	user_id	amount	salary_date	status
1	3	4500	2025-05-08	paid

Database:

```
CREATE DATABASE point_system;
USE point_system;
CREATE TABLE stops (
    Stop_id INT AUTO_INCREMENT PRIMARY KEY,
    Stop_name VARCHAR(100) NOT NULL
);
CREATE TABLE users (
    User_id INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(150) NOT NULL UNIQUE,
    Password_hash VARCHAR(255) NOT NULL,
    Role ENUM('student', 'admin', 'driver', 'transport_staff'),
    Phone VARCHAR(20),
    Created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE buses (
    Bus_id INT AUTO_INCREMENT PRIMARY KEY,
    Bus_name VARCHAR(100) NOT NULL,
    Location VARCHAR(255),
    Max_capacity INT CHECK (max_capacity >= 0),
    Current_students INT DEFAULT 0 CHECK (current_students >= 0),
    Driver_id INT UNIQUE,
    Management_staff_id INT UNIQUE,
    FOREIGN KEY (driver_id) REFERENCES users(user_id),
    FOREIGN KEY (management_staff_id) REFERENCES users(user_id)
);
CREATE TABLE routes (
    Route_id INT AUTO_INCREMENT PRIMARY KEY,
    Route_name VARCHAR(100) UNIQUE NOT NULL,
    Start_stop INT NOT NULL,
    End_stop INT NOT NULL,
    FOREIGN KEY (start_stop) REFERENCES stops(stop_id),
    FOREIGN KEY (end_stop) REFERENCES stops(stop_id)
);
CREATE TABLE route_stops (
    Route_stop_id INT AUTO_INCREMENT PRIMARY KEY,
    Route_id INT NOT NULL,
    Stop_id INT NOT NULL,
    Stop_sequence INT NOT NULL,
    UNIQUE (route_id, stop_id),
```

```

    UNIQUE (route_id, stop_sequence),
    FOREIGN KEY (route_id) REFERENCES routes(route_id) ON DELETE CASCADE,
    FOREIGN KEY (stop_id) REFERENCES stops(stop_id)
);
CREATE TABLE student_bus_registrations (
    Registration_id INT AUTO_INCREMENT PRIMARY KEY,
    Student_id INT NOT NULL UNIQUE,
    Bus_id INT NOT NULL,
    Registration_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES users(user_id),
    FOREIGN KEY (bus_id) REFERENCES buses(bus_id)
);
CREATE TABLE bus_schedule (
    Schedule_id INT AUTO_INCREMENT PRIMARY KEY,
    Bus_id INT NOT NULL,
    Route_id INT NOT NULL,
    Departure_stop_id INT,
    Arrival_stop_id INT,
    Departure_time DATETIME NOT NULL,
    Arrival_time DATETIME,
    Status ENUM('scheduled', 'cancelled', 'completed'),
    FOREIGN KEY (bus_id) REFERENCES buses(bus_id),
    FOREIGN KEY (route_id) REFERENCES routes(route_id),
    FOREIGN KEY (departure_stop_id) REFERENCES stops(stop_id),
    FOREIGN KEY (arrival_stop_id) REFERENCES stops(stop_id)
);
CREATE TABLE bus_attendance (
    Record_id INT AUTO_INCREMENT PRIMARY KEY,
    Student_id INT NOT NULL,
    Attendance_date DATE NOT NULL,
    Status ENUM('present', 'absent'),
    Check_in_time DATETIME,
    UNIQUE (student_id, attendance_date),
    FOREIGN KEY (student_id) REFERENCES users(user_id)
);
CREATE TABLE driver_attendance (
    Attendance_id INT AUTO_INCREMENT PRIMARY KEY,
    Driver_id INT NOT NULL,
    Attendance_date DATE NOT NULL,
    Status ENUM('present', 'absent'),
    Check_in_time DATETIME,
    UNIQUE (driver_id, attendance_date),
    FOREIGN KEY (driver_id) REFERENCES users(user_id)
);

```

```

);
CREATE TABLE feedback (
    Feedback_id INT AUTO_INCREMENT PRIMARY KEY,
    Student_id INT NOT NULL,
    Driver_id INT,
    Bus_id INT,
    Comments VARCHAR(1000),
    Rating TINYINT CHECK (rating BETWEEN 1 AND 5),
    Feedback_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES users(user_id),
    FOREIGN KEY (driver_id) REFERENCES users(user_id),
    FOREIGN KEY (bus_id) REFERENCES buses(bus_id)
);

CREATE TABLE maintenance_logs (
    Log_id INT AUTO_INCREMENT PRIMARY KEY,
    Bus_id INT NOT NULL,
    Description VARCHAR(1000),
    Maintenance_date DATE NOT NULL,
    Cost DECIMAL(10,2),
    Handled_by INT,
    FOREIGN KEY (bus_id) REFERENCES buses(bus_id),
    FOREIGN KEY (handled_by) REFERENCES users(user_id)
);

CREATE TABLE incident_reports (
    Report_id INT AUTO_INCREMENT PRIMARY KEY,
    Reported_by INT NOT NULL,
    Bus_id INT,
    Description VARCHAR(2000) NOT NULL,
    Report_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    Status ENUM('open', 'in_progress', 'resolved', 'closed') DEFAULT 'open',
    FOREIGN KEY (reported_by) REFERENCES users(user_id),
    FOREIGN KEY (bus_id) REFERENCES buses(bus_id)
);

CREATE TABLE notifications (
    Notification_id INT AUTO_INCREMENT PRIMARY KEY,
    User_id INT NOT NULL,
    Title VARCHAR(200),
    Message VARCHAR(2000),
    Sent_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    Read_status ENUM('unread', 'read') DEFAULT 'unread',
    FOREIGN KEY (user_id) REFERENCES users(user_id)

```

```

);
CREATE TABLE payments (
    Payment_id INT AUTO_INCREMENT PRIMARY KEY,
    Student_id INT NOT NULL,
    Amount DECIMAL(10,2) CHECK (amount >= 0),
    Payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    Payment_status ENUM('pending', 'completed', 'failed'),
    Transaction_id VARCHAR(100) UNIQUE,
    FOREIGN KEY (student_id) REFERENCES users(user_id)
);

CREATE TABLE salaries (
    Salary_id INT AUTO_INCREMENT PRIMARY KEY,
    User_id INT NOT NULL,
    Amount DECIMAL(10,2) CHECK (amount >= 0),
    Salary_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    Payment_status ENUM('pending', 'paid', 'failed') DEFAULT 'pending',
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

START TRANSACTION;
COMMIT;

```

Normalized Report for Bus System Tables

Bus Schedule Stops Table

Column Name	Data Type	Constraints	Description
schedule_stop_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique stop schedule ID
schedule_id	INT	FOREIGN KEY	References bus_schedule(schedule_id)
stop_id	INT	FOREIGN KEY	References stops(stop_id)
stop_time	DATETIME	NOT NULL	Scheduled time at the stop
type	ENUM(...)	NOT NULL	Type: 'departure' or 'arrival'

Indexes: schedule_id, stop_id

Normalization: Extracted repeated stop-time fields from bus_schedule to achieve **(1NF & 2NF)**

Bus Attendance Table

Column Name	Data Type	Constraints	Description
record_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique attendance ID
user_id	INT	FOREIGN KEY	References users(user_id)
role	ENUM(...)	NOT NULL	Role: 'student' or 'driver'
attendance_date	DATE	NOT NULL	Date of attendance
status	ENUM(...)	NOT NULL	Attendance status: present/absent
check_in_time	DATETIME		Time of check-in

Constraints: UNIQUE(user_id, attendance_date)

Indexes: user_id, attendance_date

Normalization: Merged student_attendance and driver_attendance into a single table to eliminate redundancy (**2NF, 3NF**)

Feedback Table

Column Name	Data Type	Constraints	Description
feedback_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique feedback ID
student_id	INT	FOREIGN KEY	Feedback author (user_id)
driver_id	INT	FOREIGN KEY (nullable)	Optional driver
bus_id	INT	FOREIGN KEY (nullable)	Optional bus
comments	VARCHAR(1000)		Feedback message
rating	TINYINT	CHECK (1-5)	Rating out of 5
feedback_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date of submission

Indexes: student_id, driver_id, bus_id

Normalization: Maintains clean relationships and ensures referential integrity via FKs (**3NF**)

Maintenance Logs Table

Column Name	Data Type	Constraints	Description
log_id	INT	PRIMARY KEY, AUTO_INCREMENT	Maintenance log ID
bus_id	INT	FOREIGN KEY	References buses(bus_id)
description	VARCHAR(1000)		Description of work done
maintenance_date	DATE	NOT NULL	Maintenance date
cost	DECIMAL(10,2)		Cost of maintenance
handled_by	INT	FOREIGN KEY	Staff in charge (user_id)

Indexes: bus_id, handled_by

Normalization: Handled-by field normalized using foreign key to enforce staff accountability (**2NF, 3NF**)

Incident Reports Table

Column Name	Data Type	Constraints	Description
feedback_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique feedback ID
student_id	INT	FOREIGN KEY	Feedback author (user_id)
driver_id	INT	FOREIGN KEY (nullable)	Optional driver
bus_id	INT	FOREIGN KEY (nullable)	Optional bus
comments	VARCHAR(1000)		Feedback message
rating	TINYINT	CHECK (1-5)	Rating out of 5
feedback_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date of submission

Indexes: reported_by, status

Normalization: Avoids data duplication. Separates each report with FK references for scalability (3NF)

Notifications Table

Column Name	Data Type	Constraints	Description
notification_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique notification ID
user_id	INT	FOREIGN KEY	Recipient
title	VARCHAR(200)		Notification subject
message	VARCHAR(2000)		Message content
sent_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Sent timestamp
read_status	ENUM(...)	DEFAULT 'unread'	Read status

Indexes: user_id, read_status

Normalization: Properly separates each user's notification entry. Can be extended for multirecipient via notification_recipients table

Payments Table

Column Name	Data Type	Constraints	Description
payment_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique payment ID
student_id	INT	FOREIGN KEY	Who made the payment
amount	DECIMAL(10,2)	CHECK (>= 0)	Amount paid
payment_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date of payment
payment_status	ENUM(...)	NOT NULL	Status: pending, completed, failed
transaction_id	VARCHAR(100)	UNIQUE	Unique transaction reference

Indexes: student_id, transaction_id (UNIQUE)

Normalization: Supports multiple payments per student. Atomic transaction structure (3NF)

Salaries Table

Column Name	Data Type	Constraints	Description
salary_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique salary ID
user_id	INT	FOREIGN KEY	Paid to which user
amount	DECIMAL(10,2)	CHECK (>= 0)	Salary amount
salary_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Payment date
payment_status	ENUM(...)	DEFAULT 'pending'	Status of payment

Indexes: user_id, payment_status

Normalization: Clean separation of salary transactions per user. Enforces financial structure (3NF)

DML & DDL QUERIES:

DML QUERIES:

1. Insert a new student into the users table

```
INSERT INTO users (name, email, password_hash, role, phone)
VALUES ('Ali Raza', 'ali.raza@student.neduet.edu.pk', 'hashed_password_123', 'student',
'03001234567');
```

2. Register a student to a bus

```
INSERT INTO student_bus_registrations (student_id, bus_id)
VALUES (1, 2);
```

3. Update a bus's current student count after registration

```
UPDATE buses
SET current_students = current_students + 1
WHERE bus_id = 2;
```

4. Record feedback from a student

```
INSERT INTO feedback (student_id, driver_id, bus_id, comments, rating)
VALUES (1, 5, 2, 'Very punctual and clean bus.', 5);
```

5. Mark a driver as present for today

```
INSERT INTO driver_attendance (driver_id, attendance_date, status, check_in_time) VALUES
(5, CURDATE(), 'present', NOW());
```

DDL QUERIES:

1. Add a new column gender to the users table

```
ALTER TABLE users
ADD gender ENUM('male', 'female', 'other') AFTER phone;
```

2. Modify the bus_name column in buses to allow only unique values

```
ALTER TABLE buses
ADD CONSTRAINT unique_bus_name UNIQUE (bus_name);
```

3. Rename the description column in maintenance_logs to details

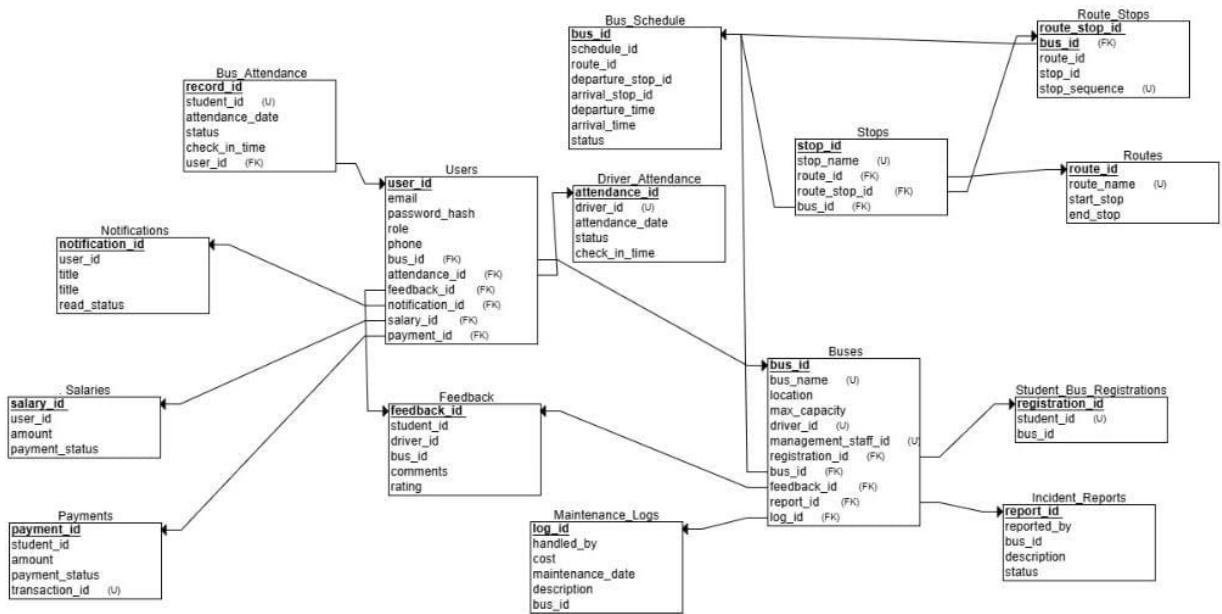
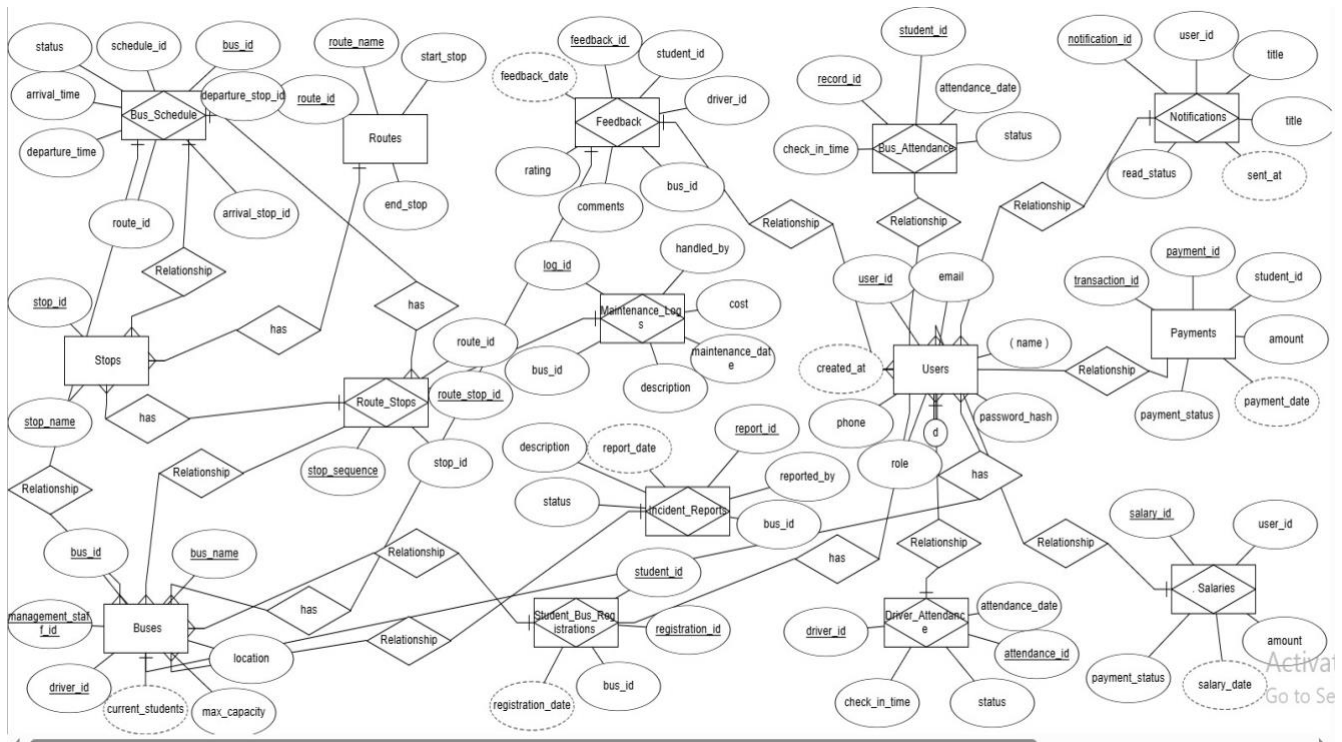
```
ALTER TABLE maintenance_logs
CHANGE description details VARCHAR(1000);
```

4. Drop the notifications table DROP TABLE notifications;

5. Create a view to show students and their assigned buses CREATE VIEW student_bus_view AS

```
SELECT u.user_id, u.name AS student_name, b.bus_name
FROM users u
JOIN student_bus_registrations sbr ON u.user_id = sbr.student_id
JOIN buses b ON sbr.bus_id = b.bus_id WHERE
u.role = 'student';
```

ERD:



Conclusion

The Point System Database provides a reliable, scalable, and efficient solution for managing transportation services in an organization. It enables administrators to manage routes, track student attendance, monitor driver performance, and maintain a complete history of maintenance and incidents. The system ensures secure payment management for students and salary management for staff. By leveraging this database, the organization can optimize its transportation operations and improve service quality.